

Audion VS Engine Lite

Windows 10 | 11

release no releases or repo not found

downloads repo not found

license repo not found

Version 2.0.1 · 2026-08-24 · 449.5 MB

- [Direct download](#) — unmetered, no rate limits
- [Project page](#) — every version and how to install

SHA-256: d12d3e13ce0873cce67865ba8b9ee0631e3f63d9104a7cc9007c5195b7e16cf0

Portable Windows toolkit for **technical video preprocessing and stylization** built on VapourSynth + FFmpeg. Lite-build = classical VS plugins, **no ML stack. 22 presets across 4 independent palettes**:

Palette	Presets	What it does
Precision Engine	8	Denoise (incl. SOTA shadow-targeted BM3D), debanding, controlled fine-grain restoration. Technical layer that goes BEFORE the colorist.
Film Looks Engine	5	Cinematic emulations: 35mm Kodak (250D/500T/50D), 16mm, Super 8, Bleach Bypass, universal "Cinematic".
Retro Engine	2	Analog character: VHS / CRT, Camcorder 90s.
Restoration Engine	7	"Things you cannot do in Adobe / DaVinci out of the box": QTGMC, TIVTC, MVTools-MCDeGrain, derainbow, deblock, dehaze, plus NNEDI3/ZNEDI3 2x non-ML upscale.

The engine is a **CLI-driven Python orchestrator** wrapping `vspipe` | `ffmpeg` pipelines. Launchers are batch files with FZF + CMD fallback (English and Russian copies).

Lite **does not** include the ML stack (Real-ESRGAN / RIFE / DPIR) — that lives in the full release `Audion VS Engine`. Lite **does not** include the 18.C-extras (`imax_70mm`, `anamorphic_scope`, `polaroid`) either. Lite is the lean baseline (~1.2 GB working set after install) for users who do not need ML.

First run: install the engine

The build ships without VapourSynth and its plugins. They are not ours to redistribute — some seventy modules, each under its own licence. The program installs them itself, from their authors, in a couple of clicks.

Until you do that, **the program will not process anything**. It starts, but the engine behind it is missing.

Run `builder_main.cmd` (or Start → the build menu) and pick, in this order:

#	Menu entry	What it installs
10	<code>VAPOURSYNTH</code>	the engine itself
11	<code>VS PLUGINS</code>	the filters the presets call

The order matters: plugins need the engine already in place. Both steps are required — there is nothing optional here.

FFmpeg and your NVIDIA driver

Newer is not always better. Every FFmpeg build is compiled against one specific version of the NVENC headers, and each of those demands a minimum driver. Put the newest build on an older driver and hardware encoding does not get faster — it stops working.

FFmpeg build	NVENC headers	Minimum NVIDIA driver (Windows)
9.0.1	ffnvcodec n13.1.15.0	610.0
8.0.1	ffnvcodec n13.0.19.0	570.0
7.1.1	ffnvcodec n13.0.19.0	570.0

FFmpeg build	NVENC headers	Minimum NVIDIA driver (Windows)
7.1	ffnvcodec n12.2.72.0	551.76

Note the third row: 7.1.1 is built with the same headers as 8.0.1, so it needs the same 570.0 — going "one version back" buys nothing on an older driver. The step that does help is 7.1 without the patch release.

This is why the installer picks a build from your driver version instead of always taking the latest. The versions above are read from the build's own README; the driver thresholds come from the nv-codec-headers README.

If you have no NVIDIA GPU, none of this applies — the latest build is installed and encoding runs on the CPU.

Which build ships with this product: 8.0.1. That is a deliberate choice, not a missed update. Most editing and encoding machines today run drivers roughly between 571 and 609; the 610 branch is installed by very few. Both 8.1.x and 9.x demand that branch — shipping them would advertise NVIDIA hardware encoding and then deny it to most of the people it was promised to. 8.0.1 has everything these products use and runs on the drivers people actually have.

First-time install (do this once before Quick start)

If the project tree was just unpacked / cloned and `system_core/vapoursynth/` / `Tools/ffmpeg/` are still empty — run `builder_main.cmd` and walk through items in this exact order:

Stage 1 – Orchestrator Python (auto-runs on first launch of any *.cmd, but you can pre-flight it):

[01] BUILD PORTABLE ENV CMD BUILDER (or [03] INSTALL PORTABLE OFFLINE)

Stage 2 – VS engine stack (this is where ALL GREEN comes from):

[04] POWERSHELL – pwsh 7 portable
 * SKIP if `pwsh -v` already shows 7+ on

system

[10] VAPOURSYNTH – latest VS stable + own embedded Python

3.12.x

[11] VS PLUGINS – vsrepo + plugins (depends on [10])

[12] FFMPEG – portable ffmpeg

Stage 3 – Verify (only meaningful AFTER stage 2):

[71] VERIFY / DOCTOR – runs doctor.py end-to-end smoke
 (live BM3D CUDA invocation if NVIDIA)

Stage 4 – Optional cache cleanup:

[70] CLEAN INSTALL CACHE – frees transient install cache/staging
 archives. Re-installing later
 re-downloads automatically.

Why running [04] earlier reports failures: doctor.py probes the full stack (vspipe, plugins, ffmpeg, optional CUDA). Before stage 2 those binaries don't exist yet — the failures are expected, not bugs.

About portable PowerShell ([10]): Windows 10/11 ships only Windows PowerShell 5.1, but our `install/*.ps1` scripts use PS 7+ syntax (ternary, null-coalescing). So **[10] is required unless `pwsh -v` already shows 7+** on the host. The `.cmd` wrappers auto-detect and prefer system pwsh 7+ over the portable copy when both exist.

Lite has **no MLRT install step**. Cache cleanup is `[70]`, not an ML install. For ML presets use the full release `Audion VS Engine`.

Quick start

1. Unpack the project anywhere on your drive (it's portable, no install needed).
2. Run `launcher_project.cmd` — top-level dispatcher: `[P]` Precision / `[F]` Film Looks / `[R]` Retro / `[N]` Restoration / `[A]` Apply profile to file or folder.
3. Drop your source media into `input\`, results go to `output\`.

Each palette launcher walks you through input → params → encoder → run.

Audio Policy

The GUI exposes video-stream encoding only. Audio is handled automatically:

- source audio is copied by default (`-c:a copy`);
- when a ProRes source is transcoded to lossy x264/x265, audio is encoded as AAC 384 kbps for container compatibility;
- `--no-audio` remains a technical CLI option for smoke/benchmark runs, not the default GUI workflow.

Four palettes — concrete content

Precision Engine (`cli\launcher_precision.cmd`)

Pipeline-ordered menu reads top-to-bottom as the actual signal flow:

```

=== Stage 1 -- Denoise (clean first) ===
[01] Mild denoise                DFTTest spectral, light/medium/strong
[02] Shadow denoise SOTA *       BM3D (CUDA->CPU auto) + smooth luma-mask "shadows
only"
[03] Chroma cleanup              DFTTest on chroma planes only

=== Stage 2 -- Deband (smooth gradients) ===
[04] Deband safe                 neo_f3kdb light, no grain
[05] Deband + fine grain         neo_f3kdb + AddGrain micro-restore

=== Stage 3 -- Compositions (full chains) ===
[06] Filmic rebuild             *      denoise -> deband -> luma-zoned grain (flagship)
[07] Archive clean              neutral master, no grain
[08] Pre-grade prep             minimum-touch handoff to DaVinci Resolve

```

Cross-vendor: `audion_lib.bm3d_auto()` probes NVIDIA via `nvidia-smi -L` and picks BM3DCUDA / BM3DCPU automatically. Intel / AMF work out of the box.

Film Looks Engine (`cli\launcher_film_looks.cmd`)

5 looks ordered by character intensity (subtle → extreme):

```

[01] Cinematic                universal subtle filmic, safe default
[02] Film 35mm                 Kodak 250D / 500T / 50D variants
[03] Film 16mm                 organic, grainier, lifted blacks
[04] Super 8                   heaviest grain, faded '70s
[05] Bleach bypass             high contrast, desaturated ('Se7en')

```

All Film Looks share `system_core/presets/audion_lib.py`: MTF softening, halation bloom, luma-zoned grain, gamma curve, black-lift, desaturation.

Retro Engine (`cli\launcher_retro.cmd`)

```

[01] VHS / CRT                 chroma bleed, soft optics, analog noise
[02] Camcorder 90s             softer than VHS, slight overexposure

```

Restoration Engine (`cli\launcher_restoration.cmd`)

The "Adobe / DaVinci cannot do this out of the box" set. Requires extra plugins (havsfunc, mvsvfunc, mvtools, tivtc, znedi3, nnedi3_resample, plus the Python `vsutil` dependency for havsvfunc) —

installed automatically via `Install-VS-Plugins.cmd`.

```
=== Field rebuild ===
[01] QTGMC deinterlace      NNEDI3 + MVTools; gold-standard deinterlace for legacy
DV/HDV/VHS
[02] TIVTC inverse-telecine  NTSC 29.97 telecined -> 23.976 progressive (3:2
pulldown removal)

=== Motion-compensated denoise ===
[03] MVTools MCDeGrain      temporal denoise that keeps detail (Topaz-style
internals)
[04] Derainbow / decross     NTSC composite chroma cleanup (rainbow, dot crawl)

=== Compression rescue ===
[05] Deblock H.264 artefacts over-compressed YouTube / WhatsApp / SD broadcast
salvage
[06] Dehaze / local contrast clarity, no halos, no color shift, 16-bit math

=== Clean upscale ===
[07] NNEDI3 2x upscale       deterministic non-ML 2x enlargement for Lite builds
```

System requirements

- Windows 10/11 x64
- ~1.2 GB free disk space (project payload after unpack — Lite, no ML stack)
- Optional: NVIDIA GPU + driver R525+ for `bm3dcuda` acceleration (10–50× speedup vs CPU on BM3D-heavy presets). Without NVIDIA, `audion_lib.bm3d_auto()` falls back to BM3DCPU automatically.
- No system Python required — embedded Python ships in `runtime/`.
- No system FFmpeg required — portable FFmpeg ships in `Tools/ffmpeg/`.

If something is missing, run `launcher_project.cmd` → `[D] Doctor` for a stack diagnosis.

Architecture (brief)

The project uses **two independent embedded Pythons**:

```
runtime/python.exe           # Orchestrator (latest Python 3.12.x)
system_core/vapoursynth/python.exe  # VS-host (latest Python 3.12.x) -- runs
vspipe + plugins
```

The orchestrator never imports VapourSynth. It calls

```
system_core/vapoursynth/Scripts/vspipe.exe as a subprocess and pipes the y4m stream into
Tools/ffmpeg/bin/ffmpeg.exe.
```

Important for VS R74+: the real plugin autoload directory comes from

`vapoursynth.get_plugin_dir()` and, in the wheel layout, lives under `Lib\site-packages\vapoursynth\plugins\`. The old `system_core\vapoursynth\vs-plugins\` folder is legacy; it may be empty and must not be used for install/status output.

```
launcher_*.cmd → runtime/python.exe system_core/main.py run \
                --palette X --preset Y --input ... --output ...
                ↓
                ↓ subprocess: vspipe -c y4m preset.vpy - | ffmpeg -i - ... output
                ↓
                ↓ all .vpy presets read params from AUDION_VS_* env vars
                ↓
                ↓ JSON report dropped in logs/{ts}__{palette}__{preset}__{stem}.json
```

Plugin set (auto-installed via `Install-VS-Plugins.cmd`):

- v1.0 set: `lsmas`, `ffms2`, `fmtconv`, `neo_f3kdb`, `addgrain` (`grain` namespace), `knlmeanscl` (`knlm`), `bm3dcpu`, `dfttest`
- `bm3dcuda` (NVIDIA acceleration; **bundled by default**, opt-out via `/NO-CUDA`)
- Restoration set: `havsfunc`, `mvsvfunc`, `mvtools`, `tivtc`, `znedi3`, `nnedi3_resample` (+ `vsutil` from PyPI for `havsfunc`)

CLI reference

```
runtime\python.exe system_core\main.py <command> [args]
```


Command	What it does
<code>info</code>	Print resolved paths, Python versions, plugin namespace count
<code>doctor</code>	Run <code>system_core/doctor.py</code> — full stack health (Pythons, vspipe, plugins, ffmpeg, optional CUDA live smoke)
<code>list-presets</code>	All 22 registered presets across 4 palettes, with default params and palette grouping
<code>list-encoders</code>	All 34 encoder profiles (software CRF / QSV / NVENC / AMF / ProRes / DNxHR)
<code>list-profiles</code>	Built-in (5) + user-defined profiles from <code>config\profiles*.json</code>
<code>materialize-profiles [--force]</code>	Write the 5 built-in profiles to <code>config\profiles\</code> as editable JSON
<code>probe --input X</code>	ffprobe summary (codec, resolution, fps, duration, audio streams, color metadata)
<code>run --palette P --preset Q --input I --output O [params...]</code>	Full processing pipeline
<code>apply-profile --name N --input I --output O [--no-audio]</code>	Run a saved profile against a single file
<code>apply-profile-batch --name N --input-dir D --output-dir E [--recursive] [--no-mirror] [--overwrite]</code>	Profile across a folder; with <code>--recursive</code> walks subfolders, mirrors source tree, skips already-processed files

Common `run` flags: `--strength {light,medium,strong}`, `--sigma <float>`, `--use-cuda 0|1`, `--grain-back <float>`, `--shadow-threshold <0..1>`, `--transition <0..1>`, `--deband-range <int>`, `--grain-shadow / --grain-mid / --grain-high <float>`, `--high-threshold <0..1>`, `--stock {250D,500T,50D}` (film_35mm), `--intensity <float>`, `--field-order {tff,bff}`, `--qtmcpreset {Faster..Placebo}`, `--output-fps {single,double}`, `--radius <int>`, `--thsad <int>`, `--blksize <int>`, `--quant1 / --quant2 <int>`, `--encoder <profile>`, `--no-audio`. Run `... main.py run --help` for the full list.

Encoder profiles (34 total, ladder 14/17/21): software `h264_crf{14,17,21}` / `h265_crf{14,17,21}` (default `h264_crf14`); Intel QuickSync `h264_qsv_q{14,17,21}` / `h265_qsv_q{14,17,21}`; NVIDIA NVENC `h264_nvenc_q{14,17,21}` / `h265_nvenc_q{14,17,21}`; AMF `h264_amf_q{14,17,21}` / `h265_amf_q{14,17,21}`; ProRes `prores_lt` / `prores_lt_mxf` / `prores_422` / `prores_422_mxf` / `prores_422hq` / `prores_422hq_mxf`; DNxHR `dnxhr_lb/sq/hq/hqx`.

Profiles (saved combinations)

Built-in profiles ship with Lite v1.0:

Name	What
<code>shadow_clean_quick</code>	Fast shadow-noise cleanup of dark digital footage
<code>filmic_warm_35mm</code>	Kodak 250D 35mm filmic look at intensity 1.0
<code>archival_master</code>	Neutral archival master, ProRes LT (or HQ for keying)
<code>resolve_handoff</code>	Minimum-touch prep, ProRes LT (MXF wrapper available) for DaVinci / Avid
<code>vhs_dreamy</code>	VHS/CRT retro at intensity 1.2

```
runtime\python.exe system_core\main.py list-profiles
runtime\python.exe system_core\main.py apply-profile --name filmic_warm_35mm ^
--input input\source.mov --output output\filmic.mp4
```

Built-ins **auto-materialize** as editable JSON in `config\profiles\` on first use of `list-profiles` / `apply-profile`. Custom profiles: copy any `config\profiles\*.json` to a new name, edit `params` / `encoder` / `description`, and it shows up automatically in `list-profiles`.

Batch over a folder:

```
runtime\python.exe system_core\main.py apply-profile-batch ^
--name filmic_warm_35mm --input-dir input\shoot_2026_05 ^
--output-dir output\shoot_2026_05_filmic --recursive --no-audio
```

Outputs are named `<stem>__<profile>.mp4`. `--recursive` mirrors the source folder tree; already-processed files are skipped (idempotent restart-safe).

Dagnostic & maintenance scripts

`builder_main.cmd` is the menu entry. Map of items:

#	Item	Underlying script
[01..09]	Build / licenses / release	template-owned
[04]	Install portable PowerShell	<code>Install-Portable-PowerShell.cmd</code>
[10..12]	Install VS / VS plugins / FFmpeg	<code>Install-Portable-*.cmd</code> , <code>Install-VS-Plugins.cmd</code>
[70] ★	Clean install cache — removes transient install downloads, staging dirs, and bytecode caches while preserving portable payloads	<code>Clean-Install-Cache.cmd</code>
[90+]	Project launcher / open install/runtime/wheels/licenses/release	—

Lite has **no** [13] VS-MLRT LEAN / [14] VS-MLRT FULL — those are full-release steps.
Lite uses [70] CLEAN INSTALL CACHE for cache cleanup.

Standalone scripts in `install/`:

- Repair-PipShims.cmd** — fixes `Scripts\*.exe` shebangs after moving the project between drives. Auto-invoked by `system_core/engine/selfheal.py` on `main.py` / `doctor.py` startup; manual run only needed if you hit `vspipe.exe` directly outside the orchestrator. Pass `/WHATIF` (or `/N`) for a dry-run.
- Bench-CUDA.cmd** — pure-pipeline (`vspipe → ffmpeg -f null`) CPU vs CUDA timing on `shadow_denoise_sota` and `filmic_rebuild`. Drag a video onto it. Output: 4 timed lines + summary table with Δ %. Reference on RTX 5070 / Ryzen 9 5900X / DCI 4K ProRes 25 s:
`shadow_denoise_sota` -35%, `filmic_rebuild` -24%.
- Bench-AllPresets.cmd** ★ — smoke-walker over all 22 presets. Drag a video onto it; usage:
`Bench-AllPresets.cmd <video> [frames=30] [cuda_mode=off|on|sweep]`. Prints **Total** / **PASS** / **FAIL** summary + JSON report at `logs\bench_all_presets_<TS>.json`. Latest smoke: **22/22 PASS** (`Frames=1`, `Cuda=off`, 2026-05-16).
- Clean-Install-Cache.cmd** — see `builder_main → [70]`.

- `Ensure-7zip.ps1` — dot-source helper: portable `7zr.exe` / `7za.exe` are installed by `builder_main → [01]/[02]` together with Python into `system_core\7zip\`. Used by `Install-Portable-VapourSynth.ps1` and `Install-Portable-FFmpeg.ps1` for fast extraction of large archives (`Expand-7zArchive` is ~3× faster than `Expand-Archive` on multi-GB ZIPs and never chokes on >2 GB).
 - `runtime\python.exe system_core\doctor.py` — full stack health, includes a live `core.bm3dcuda.BM3D(...)` invocation as the authoritative CUDA-readiness signal.
-

Portability

The project is **fully portable**. Copy-paste the whole folder to any Windows drive or machine and it runs. No installer, no system Python, no PATH/registry changes — every component (both embedded Pythons, VapourSynth host, ffmpeg, fzf, portable PowerShell 7, 7zr.exe) lives inside `system_core/`. After moving the folder, the next `runtime\python.exe system_core\doctor.py` (or any `main.py` invocation) self-heals pip-launcher shebangs automatically. The only out-of-tree dependency is the NVIDIA video driver if you want CUDA.

CUDA setup (NVIDIA only)

`bm3dcuda` is the optional accelerated BM3D plugin. Minimum:

- **NVIDIA Studio Driver R525+** (Game Ready works too; Studio is preferred) — the driver itself ships `cuda64_12.dll` and `cufft64_*.dll`, which is everything `bm3dcuda` needs at runtime.
- **CUDA Toolkit Network installer "Runtime libraries" only (~300 MB)** — needed in the few cases where the driver-bundled runtime does not match the plugin build (e.g. on Blackwell sm_120 we used Toolkit 13.2.3 to JIT-compile PTX cleanly).

Full CUDA SDK / cuDNN / TensorRT — not required.

`Install-VS-Plugins.cmd` installs `bm3dcuda` by default. Pass `/NO-CUDA` to skip it on a non-NVIDIA host. Then run `runtime\python.exe system_core\doctor.py` to verify with a live invocation.

Where CUDA shines

- **Resolution** matters: 4K gains 1.5–2× more than 1080p relatively (BM3D = O(pixels), GPU launch overhead is constant).

- **Clip length** matters: under ~5 s the JIT/setup cost is not amortized.
- **Preset weight** matters: `shadow_denoise_sota` and `filmic_rebuild` are mostly BM3D — biggest gain. `mild_denoise`, `chroma_cleanup`, `deband_*` use DFTTest / neo_f3kdb (CPU-only) — CUDA does not help.

Reference on RTX 5070 + Ryzen 9 5900X (pure denoise, no encode):

- 1080p × 32 s H.264: `shadow_denoise_sota` -22%, `filmic_rebuild` -15%
 - DCI 4K × 25 s ProRes: `shadow_denoise_sota` -**35%**, `filmic_rebuild` -**24%**
-

Project layout

```
Audion VS Engine Lite/
├─ launcher_project.cmd  launcher_project_ru.cmd      Top-level dispatcher
├─ cli/
│   └─ launcher_precision.cmd  launcher_precision_ru.cmd  Palette CLI launchers
│   └─ launcher_precision.cmd  launcher_precision_ru.cmd  Stage 1/2/3 pipeline (8
presets)
│   └─ launcher_film_looks.cmd  launcher_film_looks_ru.cmd  5 film looks
│   └─ launcher_retro.cmd  launcher_retro_ru.cmd  2 retro looks
│   └─ launcher_restoration.cmd  launcher_restoration_ru.cmd  6 restoration presets
├─ builder_main.cmd  launcher_gui.cmd  launcher_tools.cmd  Service/GUI launchers
├─ runtime/
│   └─ orchestrator (latest 3.12.x)
│       └─ system_core/
│           └─ main.py  doctor.py  CLI + diagnostics
│           └─ engine/  runner / env / probe /
logging / presets / profile / selfheal
│   └─ presets/{precision,film_looks,retro,restoration}/  21 .vpy preset files
│   └─ vapoursynth/  VS-host (own latest)
Python 3.12.x + plugins
│   └─ presets/audion_lib.py  shared preset helpers
(incl. bm3d_auto)
│   └─ ffmpeg/  Portable FFmpeg
(BtbN/Gyan GPL)
│   └─ powershell/  Portable PowerShell 7
│   └─ 7zip/7zr.exe  Portable 7-Zip CLI
│   └─ fzf.exe  FZF binary
├─ install/  Installers + diagnostic
scripts (.cmd + .ps1):
│   │
│   └─ {PowerShell,VapourSynth,FFmpeg}, Install-VS-Plugins,
│   │
│   └─ Clean-Install-Cache (~700
MB reclaim),
│   │
│   └─ Ensure-7zip (dot-source
helper for 7zr/7za),
│   │
│   └─ Repair-PipShims (auto-
invoked by selfheal),
│   │
│   └─ Bench-CUDA (pure-denoise
CPU vs CUDA bench),
│   │
│   └─ Bench-AllPresets (smoke
across all 22 presets),
│   │
│   └─ make_release_archive
```

(release zip with dev-artefact exclusions)	
└─ GitHub/	Publication-ready docs
(this folder)	
└─ config/	Defaults + user profiles
(`profiles/*.json`)	
└─ input/ output/ logs/ release/	User folders
└─ CLAUDE.md MEMORY.md	Agent context (for AI
session continuation)	

Documentation files

- **CLAUDE.md** — working contract for AI agents continuing development (root)
- **MEMORY.md** — full project state, architecture decisions, gotchas, applied fixes (root)
- **GitHub/README_EN.md** / **README_RU.md** — user-facing landing page (this file)
- **GitHub/SECURITY.md** — security policy
- **LICENSE** / **GitHub/LICENSE (GPL-3.0-or-later).md** — GPLv3 license text for the project license (**GPL-3.0-or-later**)
- **GitHub/** other files — publication metadata (release notes, project page description, one-liner)

License

Audion-authored source code, scripts, launchers, presets, and documentation are licensed as **GPL-3.0-or-later** (see **LICENSE**). Third-party tools and libraries (VapourSynth, FFmpeg, plugins, Python, wheels, PowerShell, 7-Zip, fzf) are governed by their own licenses; collect and ship them with **builder_main.cmd** → **[06] Collect release licenses** into **licenses/** and **licenses/THIRD_PARTY_NOTICES.md**.

Status: Lite v1.0 + Phase 18.B Restoration + NNEDI3 2x  — production-ready on Windows 10/11 x64. Latest local all-presets smoke: **22/22 PASS** (**Frames=1**, **Cuda=off**, 2026-05-16). Intel encoder smoke remains **22/22 runnable profiles PASS** from 2026-05-10; NVENC/AMF require matching hardware. CUDA sweep for this 22-preset point is an NVIDIA-host handoff: **Bench-AllPresets.ps1 -Frames 1 -Cuda sweep**.

Canonical Workbench labels

Workbench uses the same Audion Image Tools public vocabulary in every project. Its buttons always keep the same order and labels: **Source**, **Add file...**, **Target**, **Reset**, **Delete**, **List**.

Reset returns to project **input/output** and does not delete files; **Delete** clears the current **Source** and **Target** only after confirmation. The exact Russian labels are **Источник**, **Добавить файл...**, **Назначение**, **Сбросить**, **Удалить**, **Список**. The Workbench variants **Destination**, **Clear**, **Цель**, and **Очистить** are not used.